A Certificate-Based Branching Analysis for MaxSAT Parameterized by the Number of Clauses

Аннотация

Мы представляем пайплайн построения и проверки локальных разборов ветвления для точного решения MaxSAT. Начиная с частично заданного окружения клоз, генератор различает пустые и непустые хвосты клоз, раскрывает дополнительные переменные и ищет стратегии ветвления. Он сочетает перебор назначений, удаление доминируемых альтернатив, группировку с помощью подстановок переменных, редукции и рекурсивный анализ полученных формул. Результатом служит сертификат, который фиксирует как полноту разбора случаев, так и преобразования в каждом случае. Отдельный верификатор восстанавливает уточнения, проверяет локальные тождества для оптимума и неравенства ветвления с помощью точной рациональной арифметики.

Мы применяем этот пайплайн к определяющему оценку случаю $(3, 2)$-литерала в алгоритме Xiao со временем работы $O^*(1.2886^m)$, где m — число клоз. Сочетая сгенерированные стратегии с леммами о ветвлении из работы Xiao, локальный анализ достигает целевого основания $q = 1.2872$. Мы доказываем корректность правил сертификата и формулируем условия, при которых этот локальный результат даёт алгоритм со временем работы $O^*(1.2872^m)$. Вклад работы — пайплайн анализа, допускающий композицию и независимую проверку; улучшенная локальная оценка демонстрирует его применение.

1 Введение

Хорошее правило ветвления для точного алгоритма должно решать две задачи: сохранять оптимальное решение среди дочерних экземпляров и достаточно уменьшать экземпляр, чтобы обеспечить требуемую оценку времени работы. Обе задачи усложняются, если полезное правило требует нескольких последовательных ветвлений и редукций. Доказательство также должно охватывать все возможные локальные конфигурации, включая те, чьи клозы содержат переменные за пределами отображённого окружения.

Мы исследуем автоматизацию такого анализа для MaxSAT, используя число клоз в качестве меры. Отправной точкой служит алгоритм Xiao со временем работы $O^*(1.2886^m)$ и полиномиальной памятью [1]. Случай, определяющий его оценку, связан с переменной, которая встречается трижды положительно и дважды отрицательно. Замена разбора этого случая более эффективной локальной стратегией даёт конкретную задачу для проверки возможностей инструмента автоматического анализа: целевая оценка известна, нужное окружение мало, а остальная часть алгоритма предоставляет структурную информацию, направляющую поиск.

В нашем пайплайне разделены два дерева с разными ролями. *Дерево уточнений* различает возможные окружения клоз. Для каждого его конечного случая *дерево стратегии* описывает, как редуцировать экземпляр этого случая или выполнять в нём ветвление. Генератор строит эти деревья по короткому сценарию анализа. В каждом выбранном случае он перебирает назначения, удаляет доминируемые варианты, группирует совместимые назначения и анализирует остаточные формулы. Отдельный верификатор читает полученные деревья, не повторяя поиск.

Ключевой шаг проверки прост. Для каждой возможной комбинации истинностных значений неизвестных хвостов клоз верификатор вычисляет наилучшее локальное значение целевой функции до и после предложенного преобразования. Эти значения должны совпадать с учётом уже удовлетворённых клоз. Одна такая проверка позволяет работать с редукциями и сгруппированными ветвями через общий интерфейс. Оценки рекуррентностей проверяются отдельно от тождеств для целевой функции.

Вклад работы.

1. Мы предлагаем пайплайн, сочетающий структурные уточнения с рекурсивным поиском локальных стратегий для MaxSAT, включая доминирование назначений, группировку на основе подстановок и редукции полученных дочерних формул.
2. Мы задаём правила сертификата для покрытия случаев, локальных преобразований и накопленных уменьшений меры и доказываем корректность их композиции.
3. Мы применяем пайплайн при рациональном целевом основании $q = 1.2872$ и приводим явную успешную стратегию из сгенерированного разбора. Мы отделяем сгенерированные преобразования от импортированных результатов, используемых для вывода оценки времени работы.

1.1 Связанные работы и место нашего подхода

Автоматический анализ ветвления имеет значительную историю. Fedin и Kulikov [2] генерировали доказательства оценок времени работы на основе заданных правил упрощения и описали эксперименты для SAT, MaxSAT и MaxSAT с ограниченным числом вхождений переменных. Kulikov [3] также автоматизировал поиск правил упрощения и полезных расщеплений для SAT и MaxSAT. Подход Skjernaa [4, глава 8] уточняет шаблоны формул, ищет редукции и рассматривает ветвление как по значениям переменных, так и по равенству или различию значений пар переменных. Таким образом, автоматический разбор случаев, поиск редукций и ветвление, выходящее за рамки отдельных назначений, — уже известные идеи.

В более поздней работе Červený и Suchý [5] генерировали параметризованные алгоритмы ветвления для d-Path Vertex Cover. Gao и соавторы [6] формулируют выбор локального правила ветвления для Maximum Independent Set как взвешенную задачу покрытия множества. Эти работы также мотивируют поиск набора альтернатив вместо выбора одной переменной ветвления.

Наш подход объединяет уточнения хвостов клоз, рекурсивный анализ остаточных CNF, полученных подстановками, и отдельный верификатор покрытия и локальных тождеств для оптимума. В частности, сертификат фиксирует конкретные дочерние формулы и их композицию, поэтому добавление новой поисковой эвристики не требует отдельного правила проверки, специфичного для этой эвристики. Мы представляем такое сочетание для определяющего оценку случая Xiao, не утверждая новизну углублённого поиска, доминирования или группировки по отдельности.

Исходный алгоритм MaxSAT опирается на анализ по числу клоз в работах Bansal и Raman [7], Chen и Kanj [8], а также Xu и соавторов [9]; близкие методы резолюции рассматриваются у Chen, Xu и Wang [10]. Другие параметризации, в том числе изученные Bliznets и Golovnev [11], приводят к иным оценкам. Общие сведения о рекуррентностях ветвления приведены у Fomin и Kratsch [12].

Наконец, протоколирование доказательств для MaxSAT позволяет сертифицировать препроцессинг, сохраняющий оптимум, в том числе с помощью систем VeriPB и CakePB [13]. Наш сертификат имеет другую цель: локальный разбор случаев и его рекуррентность для семейства экземпляров. Его семантические проверки перебирают небольшие локальные

таблицы истинности, тогда как псевдобулево протоколирование доказательств поддерживает рассуждения об отдельных, потенциально больших экземплярах, обрабатываемых решателем.

2 Предварительные сведения и целевой случай

Мы рассматриваем невзвешенную задачу MaxSAT. Формула F — мультимножество клоз, а клоза — дизъюнкция литералов. Одинаковые клозы сохраняются с учётом кратности. Повторяющиеся литералы внутри клозы можно удалить; тавтологические клозы можно удалить со смещением целевой функции на единицу за каждую, а пустые клозы — с нулевым смещением. Предполагается, что такая нормализация выполнена в начале алгоритма. Обозначим через $m = |F|$ полученное число клоз и положим

$$\mathrm{OPT}(F) = \max_{\alpha} \mathrm{sat}(F, \alpha),$$

где α пробегает все назначения, а sat считает удовлетворённые клозы с учётом кратности. Мы непосредственно описываем преобразования оптимума; эквивалентная задача разрешения спрашивает, выполнено ли $\mathrm{OPT}(F) \geq k$.

Литерал x имеет степень (i, j), если он встречается в i клозах, а его дополнение $\bar{x}$ — в j клозах. Дополнительный литерал имеет степень (j, i). Назовём $(i, 1)$-литерал *singleton*, если его единственное отрицательное вхождение — единичная клоза $(\bar{x})$.

2.1 Шаблоны, локальные оценки и уменьшения меры

Шаблон P — мультимножество отображённых клоз. Выбранные переменные записываются в нём явно, а ещё не раскрытая часть клозы обозначается *хвостом* ?: хвост $?^+$ содержит хотя бы один литерал, тогда как ? может быть пустым. Обозначения описывают синтаксис клозы, а не её значение истинности: непустой хвост тоже может быть ложным. Нижние индексы различают хвосты; повторяющиеся индексы обозначают одну и ту же дизъюнкцию. В шаблоне отображаются все вхождения выбранных переменных. Клозы, записанные подряд, образуют мультимножество, поэтому одинаковые клозы учитываются с кратностью.

Полная формула *соответствует* P, если её можно записать как $F = P[T] \uplus E$. Здесь T заменяет хвосты дизъюнкциями литералов на невыбранных переменных, а E содержит остальные клозы. Символ $\uplus$ обозначает объединение мультимножеств: кратности складываются, даже если в обеих частях есть одинаковые клозы. Выбранные переменные не встречаются ни в T, ни в E. Соответствие также учитывает заданные ограничения на степени и singleton. Для булева вектора τ значений хвостов определим *локальную оценку*

$$B_P(\tau) = \max_{\alpha} \mathrm{sat}(P, \alpha, \tau), \tag{1}$$

где α задаёт значения выбранных переменных, а каждый хвост рассматривается как булев вход. Клоза, ставшая истинной за счёт хвоста, считается одной удовлетворённой клозой. В полной формуле разные хвосты могут иметь общие скрытые переменные. Проверка всех векторов τ охватывает каждую реализуемую комбинацию и потому может отклонить допустимое преобразование, но не может принять недопустимое по этой причине.

Корректный шаг ветвления порождает формулы F_i и неотрицательные целые *смещения* s_i, для которых

$$\mathrm{OPT}(F) = \max_{i} \big(s_i + \mathrm{OPT}(F_i)\big).$$

Его *уменьшения меры* равны $d_i = |F| - |F_i|$. Они не обязаны совпадать со смещениями: пустая ложная клоза вносит вклад в d_i, но не в s_i. Например, назначение $x = 1$ в $(x)(\bar{x})$ даёт $s = 1$ и $d = 2$. Генератор описывает такие шаги локальными преобразованиями шаблонов; лемма 2 связывает проверенное локальное тождество с соответствующими полными формулами F_i.

Для рационального целевого основания $q > 1$ вектор положительных целых уменьшений называется *q-допустимым*, если

$$\sum_{i=1}^{r} q^{-d_i} \leq 1. \tag{2}$$

Мы используем $q = 1.2872$ и проверяем это неравенство точно. Обозначение O^* скрывает множители, полиномиальные по длине входа.

Лемма 1 (Рекуррентность ветвления)**.** *Пусть каждый рекурсивный шаг имеет положительные уменьшения меры, удовлетворяющие* (2)*, а работа в каждом узле, включая промежуточные редукции, полиномиальна по длине входа. Тогда время работы равно* $O^*(q^m)$.

Доказательство. Индукция по m даёт оценку числа листьев q^m: у внутреннего узла не более $\sum_i q^{m-d_i} \leq q^m$ листьев-потомков. Каждый путь от корня к листу содержит не более m рекурсивных рёбер, поскольку все уменьшения положительны. Поэтому число узлов не превосходит $(m+1)q^m$. Полиномиальный объём работы в каждом узле даёт требуемую оценку. □

2.2 Определяющий оценку случай $(3, 2)$-литерала

Алгоритм Xiao применяет редукции и случаи ветвления в фиксированном порядке. Шаг 5 обрабатывает $(3, 2)$-литерал x, входящий в клозы

$$(x \vee C_1)(x \vee C_2)(x \vee C_3)(\bar{x} \vee D_1)(\bar{x} \vee D_2). \tag{3}$$

Шаг 5.4, в котором D_1 и D_2 содержат только переменные степени пять, даёт определяющий оценку вектор $(6, 6, 5, 5)$ [1, раздел 4]. Другие случаи мотивируют выбор целевого основания: например, коэффициент ветвления для $(11, 3, 3)$ приблизительно равен 1.287190881, что немного меньше 1.2872.

Генератор начинает с шаблона из пяти клоз в (3), допуская пустоту каждого отображённого хвоста. Поэтому он может встретить как целевой случай, так и случаи, покрываемые соседними шагами алгоритма. Допустимые степени вновь раскрываемых литералов входят в конфигурацию анализа. Точный корневой шаблон и список степеней приведены в разделе 6.1; их связь с упорядоченным алгоритмом Xiao рассматривается отдельно от локальной проверки в разделе 6.

3 Обзор пайплайна и пример

Дерево уточнений сначала разбирает структурные вопросы. Пуст ли выбранный хвост? Если нет, какую степень может иметь литерал в нём и где могут находиться его остальные вхождения? Каждый ответ задаёт дочерний шаблон. Затем поиск стратегии решает оптимизационную задачу: какие назначения или группы назначений покрывают оптимальное решение и дают достаточно хороший вектор ветвления? Рисунок 1 показывает, как сценарий чередует эти этапы.

```
frontier <- {root template}
for each command in the analysis script:
    if the command selects a refinement:
        replace selected unresolved templates by their children
    if the command starts a strategy search:
        for each selected unresolved template:
            if a strategy is found, record it and remove the template
require frontier empty
assemble the certificate; check coverage, identities, and vectors
```

Рис. 1: Построение и проверка сертификата. Уточнения покрывают структурные случаи; поиск стратегии закрывает случай, когда находит подходящее ветвление или редукцию. Сценарий задаёт уточнения и ограничения поиска.

Пример 1 (Сгруппированное ветвление из сертификата)**.** Рассмотрим следующий шаблон, полученный раскрытием $(3,1)$-singleton y в двух отрицательных клозах корня:

$$\begin{aligned} P = {} & (x \vee ?_1)(x \vee ?_2)(x \vee ?_3) \\ & (\bar{x} \vee y \vee ?_4)(\bar{x} \vee y \vee ?_5)(y \vee ?_6)(\bar{y}). \end{aligned} \tag{4}$$

Для четырёх назначений (x, y) достаточно двух альтернатив. Назначения $(0,0)$ и $(1,0)$ объединяются в группу, в которой $y = 0$, а x остаётся свободной переменной. При $y = 1$ изменение x с 0 на 1 не может привести к потере удовлетворённой клозы: обе клозы с отрицательным вхождением x уже удовлетворены за счёт y. Поэтому $(0,1)$ доминируется назначением $(1,1)$.

Для первой альтернативы положим

$$H = (x \vee ?_1)(x \vee ?_2)(x \vee ?_3)(\bar{x} \vee ?_4)(\bar{x} \vee ?_5)(?_6).$$

Единичная клоза $(\bar{y})$ даёт смещение 1, а все шесть клоз H остаются в рекурсивной формуле, включая клозу $(?_6)$, состоящую только из хвоста. Вторая альтернатива удовлетворяет шесть клоз и делает $(\bar{y})$ ложной, не оставляя локальных клоз. Она имеет смещение 6 и уменьшение меры 7. Следовательно, для любого булева вектора хвостов τ

$$B_P(\tau) = \max\{1 + B_H(\tau),\, 6\}. \tag{5}$$

Для полного экземпляра $F = P[T] \uplus E$ это тождество даёт

$$\mathrm{OPT}(F) = \max\{1 + \mathrm{OPT}(H[T] \uplus E),\, 6 + \mathrm{OPT}(E)\}.$$

Таким образом, внешние клозы сохраняются в обеих рекурсивных альтернативах, в том числе в альтернативе с пустой локальной формулой. Вектор равен $(1,7)$, причём при $q = 1.2872$ выполнено $q^{-1} + q^{-7} < 0.948 < 1$.

Этот пример также объясняет, зачем генератор строит остаточные формулы. Сгруппированная альтернатива может сохранять свободную переменную, поэтому дальнейшие редукции и ветвления способны улучшить её вклад в итоговый вектор. Поиск может продолжаться ниже H, а не рассматривать группу как неделимый лист.

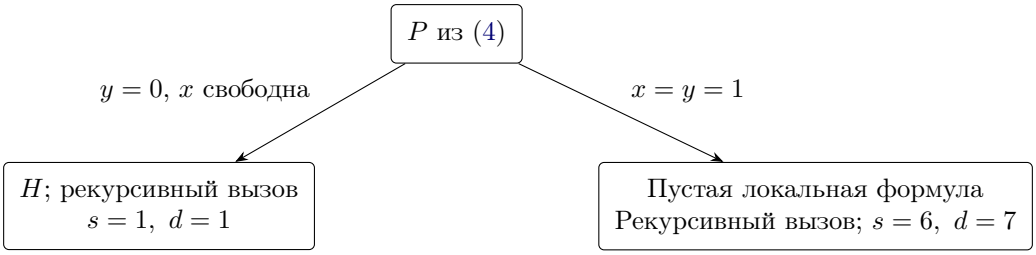


Рис. 2: Стратегия из существующего сертификата с переименованными переменными и хвостами. Внешняя часть экземпляра остаётся для дальнейшего решения даже тогда, когда локальная формула пуста.

4 Шаблоны и правила сертификата

Теперь формализуем две использованные выше проверки: уточнения покрывают нужные экземпляры, а стратегии сохраняют их оптимум. Через F обозначаем полную рекурсивную формулу, а через P — её отображённый шаблон.

4.1 Неизвестные хвосты и внешняя часть формулы

При преобразовании стратегии не каждый идентификатор хвоста родителя обязан присутствовать у потомка; каждый сохранившийся идентификатор должен принадлежать родителю. Исчезновение локальных клоз или хвостов обосновывает проверяемое тождество локальных оценок. Структурное уточнение играет другую роль. При раскрытии переменной из хвоста дочерний хвост описывает остаток после раскрытия, а отображённый фрагмент может получить дополнительные клозы из E. Соответствие всегда интерпретируется для одного и того же полного экземпляра, хотя его отображённый фрагмент меняется.

Локальная оценка $B_P(\tau)$ из (1) позволяет сравнивать такие преобразования, не строя полную формулу для каждого выбора внешних клоз.

4.2 Структурные уточнения

Уточнения различают структуру формулы, не задавая булева назначения. Две основные операции показаны на рисунке 3.



Рис. 3: Топологические узлы уточнения. Если C непусто, левый потомок — обычная, а не пустая клоза. После раскрытия одного литерала из $?^+$ оставшийся хвост обозначается $?$ и может быть пустым.

Разделение хвоста. Выбранный $?$ заменяется двумя случаями: пустым хвостом и $?^+$. Они покрывают любой конкретный хвост. Операция записывается как `tail_empty_or_nonempty`.

Раскрытие переменной. В конфигурации анализа задаётся конечный список *допустимых ориентированных степеней* (i, j) литерала, выбираемого из хвоста клозы. Первым считается число вхождений выбранного литерала, вторым — его дополнения. В нашем случае список содержит пять классов из раздела 6.1. Его полнота следует из структурных предпосылок упорядоченного алгоритма Xiao, а не из ограничения ширины клоз. Выберем литерал из непустого хвоста и обозначим его y. Для каждой степени из списка перебираются все размещения y и $\bar{y}$ в существующих хвостах клоз; для оставшихся вхождений добавляются клозы. Выбранная клоза должна содержать y; ограничения singleton требуют, чтобы единственное отрицательное вхождение было единичной клозой. Оставшийся хвост каждой затронутой клозы может быть пустым. Любая конкретная схема вхождений присутствует среди этих вариантов, если список степеней покрывает все литералы, которые могут быть выбраны.

Формат объединяет разделение хвоста и эти раскрытия в операцию `expose_at_clause`: случай пустого хвоста включается для ? и не включается для $?^{+}$. Раскрытие допускается только для шаблонов, в которых у каждой клозы не более одного хвоста и все хвосты различны. Начальное объявление переменной задаёт корневое семейство, а не уточнение произвольных формул.

Требование покрытия можно сформулировать без отдельной семантики фрагментов: *каждый полный экземпляр, соответствующий родителю и его структурным ограничениям, должен соответствовать хотя бы одному потомку.* Раскрытие y лишь переносит все её вхождения в отображённый фрагмент того же экземпляра. Верификатор заново порождает дочерние шаблоны, сравнивает их с точностью до переименования и отклоняет пропуски, лишние случаи и дубликаты. В версии 2 формата список допустимых степеней записывается в узле раскрытия. Его полнота при нужных структурных ограничениях является математической предпосылкой, а правильность перебора вхождений проверяется вычислительно.

4.3 Локальные стратегии

В сертификате четыре операции: уточнение, декомпозиция, рекурсивный вызов и применение импортированной леммы. Декомпозиции бывают двух видов.

Декомпозиция по назначению. Альтернативы задают выбранной переменной x значения 0 и 1. Верификатор упрощает обе дочерние формулы и заново вычисляет их смещения и уменьшения меры. Пусть после назначения остаётся отображённый фрагмент P_i, а мультимножество R_i клоз, состоящих только из хвостов, переносится во внешнюю часть формулы. Тогда

$$F_i = P_i[T] \uplus E \uplus R_i[T], \qquad d_i = |P| - |P_i| - |R_i|. \tag{6}$$

Удовлетворённые клозы вносят вклад в s_i, а пустые ложные клозы — только в d_i. Перенесённые клозы остаются частью рекурсивного экземпляра. Например, назначение $x = 0$ в $(x \vee ?)(\bar{x})$ переносит $(?)$ и даёт $s = d = 1$, хотя выбранных переменных больше не остаётся.

Семантическая декомпозиция. Сертификат задаёт дочерние формулы $H_1, \ldots, H_r$ и смещения s_i. Верификатор принимает преобразование, если

$$B_P(\tau) = \max_{1 \le i \le r} \big(s_i + B_{H_i}(\tau)\big) \quad \text{для каждого } \tau. \tag{7}$$

Каждая дочерняя формула использует только родительские хвосты, выбранные или новые локальные переменные. Внешняя часть формулы здесь не меняется, поэтому оставшиеся клозы, состоящие только из хвостов, должны входить в H_i, как в примере 1. Уменьшение меры равно $d_i = |P| - |H_i|$. Декомпозиция с одним потомком описывает редукцию; её

смещение также можно вывести, если $B_P - B_{H_1}$ является постоянным неотрицательным целым числом.

Лемма 2 (Замена внутри формулы)**.** *Пусть выполнено* (7)*, а локальные переменные родительской и дочерних формул нигде не встречаются во внешней части формулы или в конкретных хвостах. Тогда замена $P[T]$ альтернативами $(H_i[T], s_i)$ сохраняет оптимум $P[T] \uplus E$.*

Доказательство. Зафиксируем назначение β всем внешним переменным. Оно задаёт значения хвостов τ_β и удовлетворяет некоторое число $e(\beta)$ клоз в E. Оптимум родителя равен

$$\max_\beta \big(e(\beta) + B_P(\tau_\beta)\big) = \max_i \left(s_i + \max_\beta \big(e(\beta) + B_{H_i}(\tau_\beta)\big) \right),$$

в силу (7). Выражение справа — наилучший оптимум дочерней формулы с учётом смещения и той же внешней части формулы. □

Рекурсивный вызов и импортированная лемма. Операция `call` завершает локальный анализ и вызывает глобальный алгоритм для оставшейся полной формулы. Накопленное вдоль пути уменьшение меры становится уменьшением этого листа. Применение импортированной леммы, кодируемое как `assumption`, вместо этого задаёт именованный вектор ветвления. Верификатор записывает его имя, формулу, вектор и путь в дереве; математическое обоснование следует из цитируемого результата о ветвлении и его применимости к текущему экземпляру. Импортируются леммы 2–4 и шаг 5.1 из работы Xiao, а также рассуждение с повторным применением леммы 3 в шаге 5.5. Соответствие именам в сертификате приведено в таблице 1.

Имя в сертификате	Математическая зависимость
`lemma2`	Лемма 2 Xiao для $(i, 1)$-литерала, не являющегося singleton.
`lemma3`	Лемма 3 Xiao для переменной степени три.
`lemma4`	Лемма 4 Xiao при заданных единичных вхождениях.
`double_lemma3`	Композиция леммы 3 из шага 5.5 с сохранением второго singleton в дочерней формуле, полученной резолюцией.
`step5.1`	Случай единичной клозы для $(3, 2)$-литерала.

Таблица 1: Импортированные рассуждения о ветвлении. Имена сохранены для связи записей сертификата с работой Xiao [1]. Каждое применение требует выполнения предпосылок источника, включая альтернативу между редукцией и ветвлением и заявленные уменьшения в полном экземпляре. Верификатор записывает эти применения, а не восстанавливает их дочерние формулы или проверяет применимость.

Почему лемму 3 можно применить дважды. Рассмотрим случай ветвления в лемме 3 для $(2, 1)$-singleton y_1. В дочерней формуле, полученной резолюцией, производится замена

$$(y_1 \vee A)(y_1 \vee B)(\bar{y}_1) \quad \text{на} \quad (A)(B)$$

со смещением и уменьшением меры, равными единице. В этом случае A и B непусты; короткие клозы, для которых срабатывают редукции источника, обрабатываются до этого случая ветвления. Удаление y_1 из двух клоз сохраняет каждое вхождение любой другой переменной, поскольку кратности клоз сохраняются. В частности, другой $(2, 1)$-singleton y_2 сохраняет оба положительных вхождения и свою отрицательную единичную клозу. Когда случай ветвления леммы 3 применяется снова, два вложенных вектора $(1, 8)$ дают

$$(8,\ 1+8,\ 1+1) = (8, 9, 2).$$

Именно такая композиция используется в шаге 5.5 Xiao. Альтернативы с редукциями в импортированных леммах по-прежнему должны обрабатываться как редукции, когда они применимы; приведённое вычисление описывает случай вложенного ветвления.

4.4 Композиция и корректность

Ребро декомпозиции с уменьшением d, за которым следует дочерний вектор $(e_1, \ldots, e_t)$, вносит в охватывающую стратегию вектор $(d + e_1, \ldots, d + e_t)$. Конкатенация векторов всех альтернатив даёт её вектор листьев. Только этот накопленный вектор должен иметь положительные компоненты и удовлетворять (2); промежуточная редукция может иметь нулевое уменьшение. Потомки уточнения — отдельные структурные случаи, поэтому их векторы проверяются независимо, а не конкатенируются.

Теорема 3 (Корректность локального сертификата)**.** *Пусть конечный сертификат проходит описанные выше проверки уточнений, декомпозиций и векторов при целевом основании q. Предположим, что списки степеней при раскрытии полны при заявленных структурных ограничениях, а каждая записанная импортированная лемма допускает корректную рекурсивную стратегию с уменьшениями не меньше записанных для каждого полного экземпляра, к которому она применяется. Тогда для каждого такого экземпляра, соответствующего корню, существует локальная стратегия, сохраняющая оптимум, с q-допустимыми накопленными уменьшениями меры.*

Доказательство. Проведём индукцию по сертификату. Разделение хвоста покрывает родителя, а раскрытие переменной перечисляет все разрешённые схемы вхождений. Поэтому к каждому соответствующему экземпляру применим хотя бы один дочерний случай уточнения. Декомпозиция по назначению исчерпывающа и учитывает клозы согласно (6). Семантическая декомпозиция сохраняет оптимум по лемме 2. Вызовы завершают локальный путь, а импортированные применения используют заявленную предпосылку. Уменьшения на рёбрах телескопически складываются вдоль каждого пути; большие фактические уменьшения могут только улучшить (2). Проверка в корне каждой стратегии устанавливает требуемую рекуррентность. □

5 Автоматический поиск и реализация

Генератор на C++ поддерживает множество ещё не разобранных шаблонов. Сценарий анализа выбирает хвосты для раскрытия и моменты запуска поисковых проходов. Каждый проход сначала пробует недорогую стратегию на основе перебора, затем применяет заданный более сильный анализ к оставшимся шаблонам. Успешно разобранные случаи удаляются из рабочего множества, а их деревья доказательств записываются в журнал. Такое разделение позволяет короткому сценарию направлять структурный анализ, в то время как программа автоматически ищет локальные стратегии.

5.1 Доминирование, группировка и рекурсивный поиск

Для выбранного набора переменных генератор перебирает назначения и фиксирует, какие клозы удовлетворены, какие стали пустыми, а какие остаются неразрешёнными. Он удаляет назначение, если другой вариант не хуже при любых возможных значениях хвостов. Для этих сравнений реализация использует маски клоз и записывает их как *subsumptions*. Речь идёт о доминировании между альтернативными назначениями, а не о неограниченном удалении поглощаемых клоз CNF в MaxSAT.

Оставшиеся назначения служат кандидатами для группировки. Группа представляется заменой каждой выбранной переменной константой либо новой переменной с возможным

отрицанием:

$$x_j \longmapsto 0,\ 1,\ u_1,\ \bar{u}_1, \ldots, u_t, \bar{u}_t. \tag{8}$$

Например, $\{(0,0),(1,0)\}$ представляется как $(x,y) = (u,0)$, а $\{(0,1),(1,0)\}$ — как $(x,y) = (u,\bar{u})$. Подстановка порождает обычную остаточную CNF, к которой можно применять редукции и дальнейшее ветвление. Группа должна в точности совпадать с образом подстановки; произвольные аффинные ограничения, такие как $x_3 = u_1 \oplus u_2$, не входят в это представление.

Рекурсивный поиск по переменным пробует каждую доступную переменную ветвления и анализирует обе дочерние формулы после назначения до заданной глубины. Для каждого потомка он может остановиться на вызове или сохранить более глубокий разбор. Поэтому полезная стратегия может быть асимметричной. К остаткам группировок также применяются редукции и анализ дочерних формул; проход с группировкой включает обычный одноуровневый поиск по переменным как базовый вариант. Параметры глубины ограничивают поиск дерева назначений, а не глубину итогового глобального алгоритма.

Кандидат оценивается по накопленному вектору листьев. Коэффициенты ветвления с плавающей точкой направляют поиск на C++, а принятие сертификата использует точное рациональное целевое основание. Редукции, удаляющие клозы, могут завершаться рекурсивным вызовом. Реализованные редукции с нулевым уменьшением числа клоз устраняют переменную перед продолжением локального анализа. Ограничения глубины назначений и эти уменьшения обеспечивают конечность каждого такого локального поиска. Внешний порядок уточнений задаётся сценарием; пайплайн не претендует на полноту поиска стратегии при произвольном меньшем целевом основании.

5.2 Построение и проверка сертификата

Генератор записывает выбранные уточнения и стратегии в формате JSON Lines. Сборщик соединяет записи в одно дерево и отклоняет неразобранные случаи. Полученный JSON-сертификат читает отдельный верификатор на Python. Нормализация формул, перебор уточнений, упрощение после назначений, таблицы оптимума и рациональная арифметика векторов реализованы независимо от поиска на C++. Четырёх операций сертификата достаточно, поскольку предложенные редукции, удаления доминируемых альтернатив и сгруппированные ветви используют общее тождество (7).

Для шаблона с a локальными переменными, t хвостами и длиной представления $\ell(P)$ прямое вычисление таблицы оптимума занимает $O(2^{a+t}\ell(P))$ времени. Эта экспоненциальная работа ограничена конечными локальными шаблонами, используемыми при предварительной проверке. После фиксации сертификата его таблицы и размеры деревьев являются константами относительно размера будущего входного экземпляра.

Для таблиц значений верификатор использует целочисленные массивы NumPy. Он ограничивает каждую локальную формулу 32767 клозами, хранит значения в знаковых 16-битных массивах, а сложение с проверенными смещениями и вычисление разностей выполняет в знаковых 32-битных массивах. Неравенства ветвления проверяются точной рациональной арифметикой. Доверенная реализация включает верификатор и используемую им среду Python, JSON и NumPy; она не является формально верифицированной реализацией системы правил.

6 Сгенерированный разбор и его глобальное следствие

6.1 Корневое семейство и сценарий анализа

Корень, объявляемый в `solve.txt`, имеет вид

$$P_0 = (x \vee ?_1)(x \vee ?_2)(x \vee ?_3)(\bar{x} \vee ?_4)(\bar{x} \vee ?_5). \tag{9}$$

Все пять хвостов изначально произвольны. В этом сеансе по умолчанию используются следующие классы раскрытия:

$$(3,1)\text{-singleton}, \quad (2,2), \quad (3,2), \quad (2,3), \quad (4,1)\text{-singleton}.$$

Степени ориентированы относительно литерала, выбранного в клозе. Сеанс сначала раскрывает y в клозе, содержащей $\bar{x}$. Затем он раскрывает z в клозе, содержащей $\bar{x}$, но не y, чередуя эти уточнения с поиском стратегий. Полный сценарий и значение команд приведены в приложении A.

Завершённый локальный разбор достигает $q = 1.2872$, используя импортированные результаты из таблицы 1. Пример 1 даёт одну из его явных стратегий, не использующую импортированных применений. В полном сертификате каждый лист уточнения должен иметь стратегию, проходящую локальную проверку вектора. Именно к этому вычислительному результату применяется теорема 3.

Режим ограниченной ширины. При запуске с `-max-clause-size` k сертификат доказывает результат только для объявленного локального семейства шаблонов. Список степеней, записанный в каждом раскрытии переменной, является явной предпосылкой этого семейства: перебор размещений доказывает покрытие относительно списка, но не полноту списка для произвольного k-MaxSAT. Ограничение ширины клозы не ограничивает степень переменной, а предел маски в 30 бит является только техническим пределом локального поиска. Для глобальной оценки k-MaxSAT нужно дополнительно дать упорядоченную обработку всех пропущенных степеней и структурных случаев, сохраняющий ширину рекурсивный алгоритм, терминальный алгоритм и соответствующие рекуррентности. В режиме ограниченной ширины генератор отключает именованные assumptions, а верификатор отклоняет их, поэтому принятый ограниченный сертификат содержит только явные проверенные переходы.

6.2 От локальной стратегии к оценке времени работы

Мы сохраняем редукции Xiao и все упорядоченные случаи, кроме шага 5.4. Их связь со сгенерированным разбором можно описать явно.

Покрытие. К началу шага 5 предшествующие редукции и шаги 1–4 алгоритма Xiao оставляют только $(2,2)$-переменные, $(3,2)$-переменные, $(3,1)$-singleton и $(4,1)$-singleton [1, раздел 4.6]. Отрицательный литерал singleton встречается только в собственной единичной клозе, поэтому он не может быть скрытым литералом в клозе с выбранной переменной. Ориентация вновь раскрываемого литерала относительно выбранной клозы даёт в точности пять классов из раздела 6.1. Исходные пять вхождений x имеют вид (9). Следовательно, каждый экземпляр, достигший шага 5.4, соответствует корню при этих ограничениях. Структурные уточнения лишь раскрывают тот же экземпляр, поэтому ограничения на степени сохраняются во всём дереве уточнений.

Остальные рекуррентности. В таблице 2 проверены различные векторы вне шага 5.4 из глобальной таблицы случаев Xiao. Для вектора $(d_1, \ldots, d_r)$ последний столбец содержит значение $\sum_i q^{-d_i}$ при $q = 1.2872$; значение не больше единицы подтверждает (2). Например, для $(11,3,3)$ получаем $q^{-11} + 2q^{-3} \approx 0.99997522 < 1$. Каждая проверка использует рациональную арифметику; показанные числа — верхние границы, округлённые вверх до шести знаков после запятой. Следовательно, шаги 5.2 и 6.3 удовлетворяют целевому основанию. Завершающий шаг 7 по лемме 5 Xiao занимает $O^*(1.2576^m)$ времени, что также укладывается в эту оценку.

Шаги Xiao	Вектор ветвления	Верхняя граница $\sum_i q^{-d_i}$
1, 5.3, 6.2	$(6, 1)$	0.996729
2	$(3, 3)$	0.937761
3	$(8, 1)$	0.909569
4	$(5, 2)$	0.886532
5.1, 6.1	$(4, 2)$	0.967807
5.2, 6.3	$(11, 3, 3)$	0.999976
5.5	$(11, 10, 4, 3)$	0.975442
6.4	$(18, 18, 11, 11, 11, 11, 4, 4)$	0.998639

Таблица 2: Неизменённые случаи при $q = 1.2872$. Векторы взяты из работы Xiao [1, таблица 3]; рациональные проверки выделяют шаг 5.4 как случай, требующий замены.

Использование фиксированного сертификата. Завершённый сертификат фиксирован независимо от размера входа. Уточнение проверяет, пуст ли конкретный хвост, либо выбирает из него литерал, просматривает его вхождения и переходит к соответствующему потомку. Даже перебор всех отображений фиксированного числа отображённых переменных занимает полиномиальное время по длине входа. Записанные назначения и подстановки выполняются над конкретными клозами; новые локальные переменные выбираются вне переменных экземпляра. Клозы, состоящие только из хвостов, остаются в рекурсивной формуле согласно (6). Стратегия имеет фиксированную конечную глубину и заканчивается вызовами с положительными накопленными уменьшениями. Импортированные подпрограммы используют полиномиальные операции исходного алгоритма. Это обеспечивает полиномиальные время и память для локального сопоставления, преобразований и выбора следующих вызовов; весь поиск сертификата выполняется заранее.

Теорема 4 (Глобальное следствие)**.** *Пусть завершённый сертификат при $q = 1.2872$ удовлетворяет предпосылкам теоремы 3, а импортированные результаты применяются при выполнении условий источника. Замена шага 5.4 Xiao записанными в сертификате стратегиями даёт решение* MaxSAT *за время $O^*(1.2872^m)$ с полиномиальной памятью.*

Доказательство. Следуем упорядоченному алгоритму Xiao, используя записанную локальную стратегию при достижении заменяемого случая. Покрытие допускает такой выбор, а теорема 3 даёт его корректность и неравенство ветвления. Остальные случаи удовлетворяют тому же целевому основанию по таблице 2 и оценке для шага 7. Описанный выше выбор действий по фиксированному сертификату и лемма 1 ограничивают работу при ветвлении. Полиномиальная работа в листе может быть заменена подпрограммой шага 7: лист с оставшейся мерой m' имеет стоимость $O^*(1.2576^{m'}) \le O^*(q^{m'})$, а неравенства ветвления ограничивают сумму весов таких листьев величиной q^m. Обход в глубину имеет полиномиальную глубину и хранит полиномиальный объём информации на каждом уровне. Вместе с полиномиальной памятью завершающей подпрограммы это доказывает оценку памяти. □

Импортированные леммы — математические результаты, используемые этим алгоритмом. Их кодирование в сертификате определяет границы машинной проверки: явные переходы и арифметика рекуррентностей проверяются, а импортированные применения записываются для рассмотрения с учётом условий источника.

6.3 Воспроизведение анализа

Сеанс генерации запускается из корня репозитория после сборки исполняемого файла на C++. Следующие команды генерируют, собирают и проверяют сертификат; последнюю команду можно выполнить и для существующего сертификата, не повторяя генерацию.

```
make
./maxsat_prover test < solve.txt
python3 stitch_certificate.py pre_certificate.jsonl proof_tree.json \
  --numerator 12872 --denominator 10000
python3 verify_certificate.py proof_tree.json imported_lemmas.json
```

Генерация пересоздаёт `pre_certificate.jsonl`. Проверка записывает импортированные применения в `imported_lemmas.json`. Если этот журнал пуст, результат проверки — `ACCEPT`; иначе выводится

`ACCEPT RELATIVE TO IMPORTED LEMMAS.`

Приложенный к этой версии артефакт создан при целевом основании 1.2872 и собран как сертификат версии 2. Его структура и измеренные затраты на сборку и проверку приведены в таблице 3. Время измерено в одном запуске на AMD Ryzen 7 5800U с 16 аппаратными потоками и 15 GiB памяти с использованием Python 3.10.6, NumPy 2.2.6 и GCC 15.2.1. Генерация заняла 38.011 секунды по настенным часам (37.53 секунды пользовательского и 0.09 секунды системного времени, загрузка процессора 98%). Затраты на сборку и проверку можно повторно измерить по сохранённому журналу JSON Lines.

Свойство артефакта	Значение
Журнал генерации	5,448 записей; 9,381,911 байт
Сертификат	`maxsat-local-proof-v2`; 12,587,834 байта
Дерево	37,054 узла; максимальная глубина 8
Уточнения	60 узлов: 40 разделений хвоста и 20 раскрытий переменной
Стратегии	7,128 корней
Декомпозиции	12,765 узлов: 8,402 семантических и 4,363 по назначению
Листья	15,612 вызовов и 1,489 применений импортированных лемм
Наибольшая локальная формула	13 клоз
Генерация	38.011 секунды; 37.53 пользовательского и 0.09 системного времени
Сборка	8.75 секунды; пиковое потребление памяти 129.2 MiB
Проверка	30.72 секунды; пиковое потребление памяти 185.4 MiB

Таблица 3: Статистика сгенерированного сертификата. Верификатор сообщает `ACCEPT RELATIVE TO IMPORTED LEMMAS`.

Импортированные применения включают 1,330 вызовов `lemma3`, 119 вызовов `lemma2`, 26 вызовов `lemma4`, девять вызовов `double_lemma3` и пять вызовов `step5.1`. Для точной идентификации артефакта начальные фрагменты хешей SHA-256 имеют вид

```
pre_certificate.jsonl  59656a240d59b0613f84b5d864a2e4bc...
proof_tree.json         01d18e63e7aa0a22a77670d8ea414bb8...
```

Полные хеши записаны в `artifact_manifest.txt`, а применения импортированных лемм перечислены в `imported_lemmas.json`.

7 Заключение

Мы разработали пайплайн, который превращает частично заданное окружение MaxSAT в проверенный локальный разбор ветвления. Структурные уточнения покрывают возможные окружения, а поиск стратегий сочетает доминирование, группировку, редукции и рекурсивный анализ остаточных формул. Сертификат явно фиксирует эти этапы, сводя их проверку к покрытию, локальным тождествам для оптимума и точной арифметике рекуррентностей.

Случай $(3, 2)$-литерала демонстрирует, как пайплайн достигает целевого основания 1.2872 и позволяет уточнить анализ Xiao. Теорема о локальной корректности и глобальное следствие определяют математические зависимости, необходимые для использования сгенерированного дерева. Такое же разделение позволяет расширять поиск, сохраняя общий и доступный для проверки интерфейс.

A Сценарий анализа

Полный сеанс генерации из `solve.txt`:

```
set 1.2872

add x 3 2 a
addpos y
-x

branch
addpos z
-x
y
xiao_branch 1
xiao_branch 2
branch
empty_divide -1
branch

print 0
exit
```

Команда `add` объявляет корень; `a` разрешает произвольные хвосты. Каждая команда `addpos` читает имя, обязательные и исключённые литералы клозы; пустые строки разделяют эти данные. Команда различает пустые хвосты и перебирает раскрытия переменной. `branch` запускает перебор назначений и группировку. Две команды `xiao_branch` задают рекурсивный поиск по переменным глубины один и два. `empty_divide -1` уточняет подходящие хвосты в рабочем множестве неразобранных шаблонов, а последняя команда `branch` разбирает полученные случаи. Генератор выбирает локальные стратегии; сценарий задаёт порядок уточнений и ограничения поиска.

Список литературы

[1] Mingyu Xiao. An exact MaxSAT algorithm: Further observations and further improvements. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, pages 1887–1893, 2022.

[2] Sergey S. Fedin and Alexander S. Kulikov. Automated proofs of upper bounds on the running time of splitting algorithms. In *IWPEC*, volume 3162 of LNCS, pages 248–259, 2004. doi:10.1007/978-3-540-28639-4_22.

[3] Alexander S. Kulikov. Automated generation of simplification rules for SAT and MAXSAT. In *SAT*, volume 3569 of LNCS, pages 430–436, 2005. doi:10.1007/11499107_35.

[4] Bjarke Skjernaa. *Exact Algorithms for Variants of Satisfiability and Colouring Problems*. PhD thesis, University of Aarhus, BRICS DS-04-5, 2004. Chapter 8. `https://www.brics.dk/DS/04/5/BRICS-DS-04-5.pdf`.

[5] Radovan Červený and Ondřej Suchý. Generating faster algorithms for d-Path Vertex Cover. In *WG*, pages 157–171, 2023. doi:10.1007/978-3-031-43380-1_12.

[6] Xuan-Zhao Gao, Yi-Jia Wang, Pan Zhang, and Jin-Guo Liu. Automated discovery of branching rules with optimal complexity for the Maximum Independent Set problem. arXiv:2412.07685, 2024. https://arxiv.org/abs/2412.07685.

[7] Nikhil Bansal and Venkatesh Raman. Upper bounds for MaxSat: Further improved. In *ISAAC*, pages 247–258, 1999.

[8] Jianer Chen and Iyad A. Kanj. Improved exact algorithms for MAX-SAT. *Discrete Applied Mathematics*, 142(1–3):17–27, 2004.

[9] Chao Xu, Wenjun Li, Yongjie Yang, Jianer Chen, and Jianxin Wang. Resolution and domination: An improved exact MaxSAT algorithm. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, pages 1191–1197, 2019.

[10] Jianer Chen, Chao Xu, and Jianxin Wang. Dealing with 4-variables by resolution: An improved MaxSAT algorithm. *Theoretical Computer Science*, 670:33–44, 2017.

[11] Ivan Bliznets and Alexander Golovnev. A new algorithm for parameterized MAX-SAT. In *IPEC*, pages 37–48, 2012.

[12] Fedor V. Fomin and Dieter Kratsch. *Exact Exponential Algorithms*. Springer, 2010.

[13] Hannes Ihalainen, Andy Oertel, Yong Kiam Tan, Jeremias Berg, Matti Järvisalo, Magnus O. Myreen, and Jakob Nordström. Certified MaxSAT preprocessing. In *IJCAR*, volume 14739 of LNCS, pages 396–418, 2024. doi:10.1007/978-3-031-63498-7_24.